Experiment No. 10

Character, Digit, or Face Classification using Convolutional Neural Networks (CNNs)

Aim

To design, train and evaluate a Convolutional Neural Network (CNN) for handwritten digit classification using the MNIST dataset, and to analyse its performance using Accuracy, Precision, Recall, F1-Score and the Confusion Matrix.

Software Requirements

- Python 3, TensorFlow/Keras, NumPy, Matplotlib, scikit-learn (OpenCV optional)
- Google Colab / Jupyter Notebook / VS Code
- Dataset: MNIST (60,000 training + 10,000 test grayscale images, 28x28 pixels, 10 classes: digits 0-9)

Procedure (Steps)

- **Step 1:** Select MNIST as the dataset (digit classification).
- **Step 2:** Import libraries and load the dataset with `keras.datasets.mnist.load_data()`.
- **Step 3:** Preprocess: normalize pixels to [0,1], reshape to (28,28,1), and split training data into train (54,000) and validation (6,000) sets; test set (10,000) is kept aside.
- **Step 4:** Build the CNN: Conv2D -> MaxPool -> Conv2D -> MaxPool -> Flatten -> Dropout -> Dense(ReLU) -> Dense(10, Softmax).
- **Step 5:** Compile with Adam optimizer (learning rate 0.001), sparse categorical cross-entropy loss, batch size 128, 10 epochs.
- **Step 6:** Train the model and plot training/validation accuracy and loss curves.
- **Step 7:** Evaluate on test data: Accuracy, Precision, Recall, F1-Score, Confusion Matrix.
- **Step 8:** Predict on unseen test images and compare predicted vs. actual labels.
- **Step 9:** Analyse misclassifications (e.g. 4 vs 9, 3 vs 5, 7 vs 1) and factors affecting accuracy.
- **Step 10:** Summarize results and applications.

Program (TensorFlow / Keras)

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import (classification_report, confusion_matrix,
                             ConfusionMatrixDisplay, accuracy_score)

# 1-2. Load dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

# 3. Preprocessing: normalize, reshape, train/validation split
x_train = x_train.astype("float32") / 255.0
x_test  = x_test.astype("float32") / 255.0
x_train = np.expand_dims(x_train, -1)      # (60000, 28, 28, 1)
x_test  = np.expand_dims(x_test, -1)

x_val, y_val = x_train[-6000:], y_train[-6000:]
x_train, y_train = x_train[:-6000], y_train[:-6000]

# 4. CNN architecture
model = keras.Sequential([
```

```
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dropout(0.5),
    layers.Dense(128, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
model.summary()

# 5. Compile
model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

# 6. Train
history = model.fit(x_train, y_train, epochs=10, batch_size=128,
                    validation_data=(x_val, y_val))

# Accuracy / loss curves
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Train")
plt.plot(history.history["val_accuracy"], label="Validation")
plt.title("Accuracy"); plt.xlabel("Epoch"); plt.legend()
plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Train")
plt.plot(history.history["val_loss"], label="Validation")
plt.title("Loss"); plt.xlabel("Epoch"); plt.legend()
plt.tight_layout(); plt.show()

# 7. Evaluation on test set
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print("Test Accuracy:", test_acc)

y_pred = np.argmax(model.predict(x_test), axis=1)
print(classification_report(y_test, y_pred, digits=4))  # P, R, F1

cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(cm).plot(cmap="Blues")
plt.title("Confusion Matrix"); plt.show()

# 8. Predictions on unseen images
plt.figure(figsize=(10, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i].squeeze(), cmap="gray"); plt.axis("off")
    plt.title(f"P:{y_pred[i]} A:{y_test[i]}")
plt.tight_layout(); plt.show()

# 9. Show misclassified images
wrong = np.where(y_pred != y_test)[0][:10]
plt.figure(figsize=(10, 4))
for k, i in enumerate(wrong):
    plt.subplot(2, 5, k + 1)
    plt.imshow(x_test[i].squeeze(), cmap="gray"); plt.axis("off")
    plt.title(f"P:{y_pred[i]} A:{y_test[i]}")
plt.tight_layout(); plt.show()
```

Observations and Result

Run the program and record your own values in the table below (typical results for this architecture on MNIST are around 98-99% test accuracy; exact values vary per run).

Metric	Value (fill from your run)
Training accuracy / loss	
Validation accuracy / loss	
Test accuracy	
Macro Precision / Recall / F1	

Most confused pairs (from confusion matrix)	

Result: The CNN successfully learned hierarchical features from digit images and classified unseen digits with high accuracy. Remaining errors occurred mainly between visually similar digits (such as 4/9, 3/5, 7/1) and poorly written samples.

Answers to Questions

Q1. What is image classification? How does it differ from object detection and image segmentation?

Image classification assigns a single label (or class) to an entire image, e.g. "this image is a cat" or "this digit is 7". Object detection and segmentation go further:

Task	Output	Example
Image Classification	One class label per image	Image contains a dog
Object Detection	Class labels + bounding boxes for each object	Dog at (x,y,w,h), ball at (x,y,w,h)
Image Segmentation	Class label for every pixel (semantic) or per object instance (instance segmentation)	Exact pixel outline of the dog

Q2. Explain the architecture and working principle of a CNN.

A CNN is a deep neural network designed for grid-like data such as images. A typical architecture is:

Input -> [Convolution -> ReLU -> Pooling] x N -> Flatten -> Fully Connected -> Softmax output

- **Convolution layers** slide small learnable filters (kernels) over the image to produce feature maps. Early layers detect edges and textures, deeper layers detect shapes and object parts.
- **Activation (ReLU)** adds non-linearity.
- **Pooling** reduces spatial size and gives some translation invariance.
- **Fully connected layers** combine the extracted features for decision making.
- **Softmax output** gives class probabilities.

Working principle: During forward propagation the image passes through the layers to give predicted probabilities. The loss (cross-entropy) compares them with the true label, and backpropagation with an optimizer (e.g. Adam) updates the filter weights. Key ideas are *local receptive fields*, *weight sharing* (the same filter is applied across the image, reducing parameters) and *hierarchical feature learning*.

Q3. Role of Convolutional, Pooling and Fully Connected layers.

Layer	Role
Convolutional	Extracts local features (edges, corners, textures) using learnable filters; produces feature maps. Uses weight sharing, so it needs far fewer parameters than a dense layer.
Pooling (Max/Average)	Down-samples feature maps, reducing computation and overfitting; makes features robust to small shifts and distortions.
Fully Connected	Flattens high-level features and combines them to learn the final non-linear decision boundary; the last FC layer (with Softmax) outputs class scores.

Q4. Why is image normalization performed before training?

- Pixel values (0-255) are scaled to a small range such as [0,1] or [-1,1], keeping inputs on a similar scale.
- Gradients stay stable, so **training converges faster** and a larger learning rate can be used.
- Prevents large activations that can saturate neurons or cause exploding/vanishing gradients.

- Makes images from different sources/lighting conditions more consistent, which improves generalization.

Q5. Purpose of activation functions such as ReLU and Softmax.

- **ReLU:** f(x) = max(0, x). Introduces non-linearity so the network can learn complex patterns (without it, stacked layers collapse into a single linear map). It is cheap to compute and reduces the vanishing-gradient problem. Used in hidden layers.
- **Softmax:** converts the output scores (logits) into probabilities that are positive and sum to 1: $\text{softmax}(z_i) = e^{z_i} / \text{sum}_j\, e^{z_j}$. Used in the output layer of multi-class classification; the class with the highest probability is the prediction.

Q6. What is a Confusion Matrix? How is it used?

A confusion matrix is an N x N table in which each **row** represents the actual class and each **column** the predicted class (or vice versa). Diagonal entries are correct predictions; off-diagonal entries show which classes are being confused. For binary classification:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

It is used to compute Accuracy, Precision, Recall and F1-score, and to identify class-specific errors (for example, "4" often predicted as "9"), which guides model improvement.

Q7. Differentiate Accuracy, Precision, Recall and F1-Score with examples.

Metric	Formula	Meaning
Accuracy	(TP+TN) / (TP+TN+FP+FN)	Fraction of all predictions that are correct. Misleading on imbalanced data.
Precision	TP / (TP+FP)	Of all predicted positives, how many are truly positive.
Recall	TP / (TP+FN)	Of all actual positives, how many were found.
F1-Score	2*P*R / (P+R)	Harmonic mean of precision and recall.

Example: A test set has 100 images: 90 are not-"7" and 10 are "7". The model finds 8 sevens correctly (TP=8), misses 2 (FN=2), and wrongly labels 4 other images as "7" (FP=4), so TN=86.

- Accuracy = (8+86)/100 = 0.94
- Precision = 8/(8+4) = 0.667
- Recall = 8/(8+2) = 0.80
- F1 = 2(0.667)(0.80)/(0.667+0.80) = 0.727

Accuracy looks high (94%), yet precision is only 66.7%, which shows why the other metrics are needed. Precision matters when false alarms are costly (spam filtering); recall matters when missing a case is costly (disease detection).

Q8. Traditional feature-based methods (SIFT/HOG + SVM) vs CNN.

Aspect	SIFT/HOG + SVM	CNN
Feature extraction	Handcrafted, designed by experts	Learned automatically from data
Pipeline	Separate stages: features, then classifier	End-to-end training

Accuracy on complex data	Lower; limited by feature quality	Much higher on large, varied datasets
Data requirement	Works with small datasets	Needs large data (or transfer learning)
Computation	Lightweight; CPU is enough	Heavy; GPU preferred for training
Robustness	Sensitive to pose, lighting, background	Learns invariances, more robust
Interpretability	Easier to interpret	Black-box, harder to interpret

Q9. Five real-world applications of CNN-based image classification.

- **Handwritten digit/character recognition** - cheque processing, postal code sorting, OCR.
- **Face recognition and biometric authentication** - phone unlock, attendance systems, security.
- **Medical diagnosis** - detecting pneumonia, tumours or diabetic retinopathy from X-ray, CT, MRI and retinal images.
- **Autonomous vehicles** - recognising traffic signs, pedestrians and road conditions.
- **Agriculture and industry** - plant disease identification, crop monitoring and defect detection in manufacturing.

(Other examples: content moderation, wildlife monitoring, retail product recognition, satellite image classification.)

Q10. How do Data Augmentation, Transfer Learning and Hyperparameter Tuning improve performance?

- **Data Augmentation:** creates new training samples by random rotation, flipping, shifting, zooming, brightness change, etc. It increases dataset diversity, reduces overfitting and improves generalization. Example: `ImageDataGenerator` or `RandomRotation` layers in Keras.
- **Transfer Learning:** reuses a network pre-trained on a large dataset (e.g. ResNet, VGG, MobileNet trained on ImageNet) and fine-tunes it on the new task. It needs less data and training time and usually gives higher accuracy, especially for small datasets.
- **Hyperparameter Tuning:** searching for the best learning rate, batch size, number of filters/layers, dropout rate, optimizer and epochs (using grid search, random search, Keras Tuner or Bayesian optimization). Well-chosen hyperparameters improve convergence and balance underfitting against overfitting.

Conclusion

A CNN was designed, trained and evaluated for MNIST digit classification. Convolution and pooling layers automatically extracted useful features, and the model achieved high accuracy on unseen test data. The confusion matrix and classification report showed class-wise performance and the main sources of error (visually similar digits). Performance can be improved further with data augmentation, regularization (dropout), transfer learning and hyperparameter tuning.